

Министерство науки и высшего образования Российской Федерации
Балтийский государственный технический университет «Военмех»

*О.В. СКУЛЯБИНА, Т.В. ЛОГУНОВА,
Е.А. МАЖАЙЦЕВ*

КОМПЬЮТЕРНЫЙ ПРАКТИКУМ

Санкт-Петербург
2021

УДК 004.42(076)
С 46

С 46 **Скулябина, О.В.**
Компьютерный практикум / Ю.В. Скулябина,
Т.В. Логунова, Е.А. Мажайцев; Балт. гос. техн.
ун-т. – СПб., 2021. – 48 с.

Практикум включает пять работ, выполняемых на компьютере и расположенных в порядке возрастающей сложности. Каждая работа содержит краткие теоретические сведения, порядок выполнения, варианты индивидуальных заданий и контрольные вопросы.

Предназначено для студентов, изучающих язык программирования С или С++.

УДК 004.42(076)

Р е ц е н з е н т канд. техн. наук, доц. каф. системного анализа
и информационных технологий Санкт-Петербургского гос. технол.
ин-та (техн. ун-та) *И.В. Ананченко*

*Утверждено
редакционно-издательским
советом университета*

© Авторы, 2021
© БГТУ, 2021

ПРЕДИСЛОВИЕ

Предлагаемый «компьютерный практикум» направлен на выработку у студентов начальных навыков работы в командной строке и применения компиляторов и утилит GNU Compiler Collection. Наличие таких навыков способствует формированию профессиональных компетенций, способности разрабатывать программное обеспечение, необходимое для решения задач управления и обработки информации на таких языках, как C++, Objective-C, Java, Фортран, Ada, Go, GAS и D.

Каждую программу в работающем виде (после отладки и тестирования) студент должен показать преподавателю, после чего лабораторная работа подлежит защите.

К защите должен быть подготовлен отчет, включающий в себя, как правило, титульный лист, формулировку задания, описание исходных и результирующих данных и вспомогательных переменных, алгоритм решения задачи, текст программы и результаты ее тестирования.

Защита лабораторной работы состоит из двух частей: практической и теоретической. В практической части студент должен объяснить принципы работы одной из представленных им программ, в теоретической – ответить на вопросы по теме работы. При подготовке к защите рекомендуется подготовить ответы на контрольные вопросы, приведенные в конце каждой работы.

В конце практикума приведен список литературы, рекомендуемой для более глубокого освоения рассматриваемой темы.

ЛАБОРАТОРНАЯ РАБОТА № 1

КОМАНДНАЯ СТРОКА И ОСОБЕННОСТИ ИСПОЛЬЗОВАНИЯ НАБОРА КОМПИЛЯТОРОВ И УТИЛИТ GCC

Командная строка – это текстовый интерфейс, в котором команды отдаются путем ввода текстовых строк с клавиатуры. Другое название командной строки – консоль. Она представляет собой отдельную программу, позволяющую пользователю непосредственно взаимодействовать с операционной системой.

Чтобы запустить командную строку (рис. 1), необходимо нажать сочетание клавиш <Win>+<R> и в появившемся окне *Выполнить* ввести *cmd*. Сама утилита командной строки лежит обычно в папке *C:\Windows\System32\cmd.exe*.

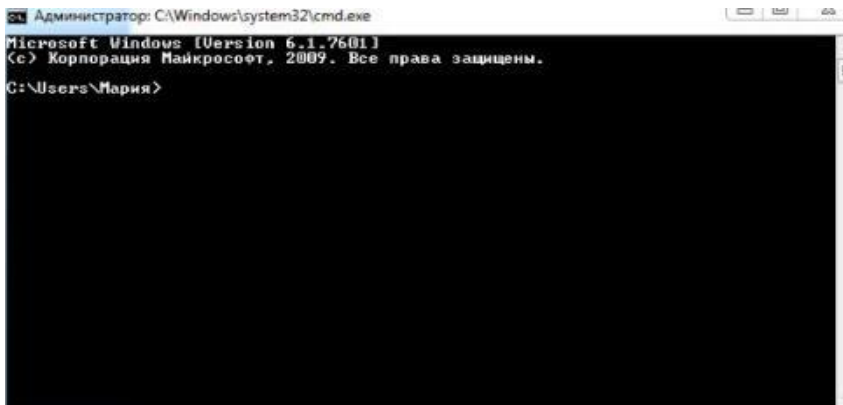


Рис. 1. Командная строка Windows

Чтобы уменьшить возможные ошибки при выполнении работ, командную строку следует запускать **от имени администратора**.

В открывшемся окне сверху расположен традиционный знак версии командной строки, а чуть ниже – строка с текущим каталогом и мигающий курсор, с которого можно начать ввод команд.

В командной строке есть горячие клавиши, наиболее часто используемые в работе:

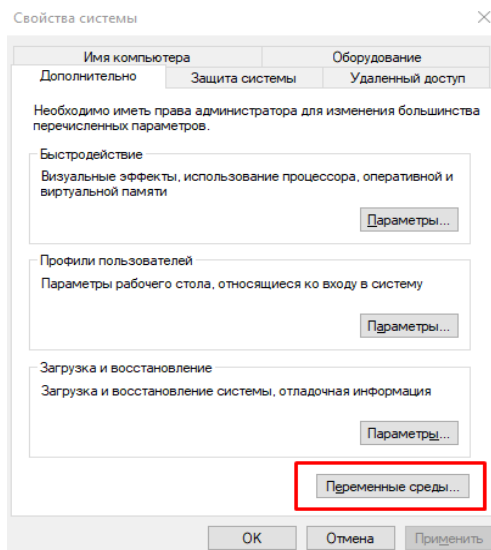


Рис. 3. Вкладка *Переменные среды*

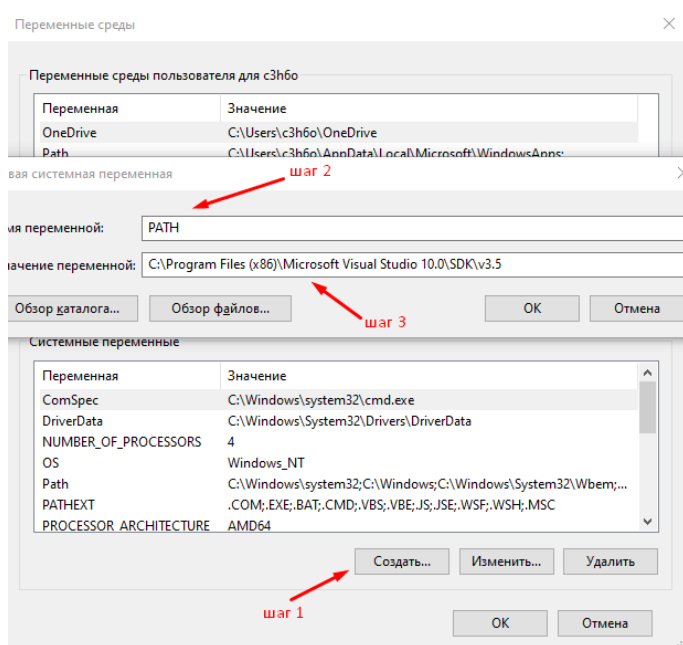


Рис. 4. Настройка системных переменных

```

C:\Users\Evgeny\Desktop>dir
Том в устройстве C не имеет метки.
Серийный номер тома: BC80-7C0F

Содержимое папки C:\Users\Evgeny\Desktop

28.11.2020  18:28    <DIR>          .
28.11.2020  18:28    <DIR>          ..
05.11.2020  15:52                223 Apex Legends.url
11.09.2020  11:55                2 238 Discord.lnk
05.10.2020  21:12                222 Dying Light.url
20.11.2020  17:59    <DIR>          electre
14.09.2020  12:08                1 144 FastStone Capture.lnk
19.10.2020  10:46    <DIR>          maga
27.11.2020  17:29    <DIR>          nop
22.07.2020  13:10                1 639 vegas170.lnk

```

Рис. 5. Результат выполнения команды *dir*

```

C:\Users\Evgeny\Desktop>

```

Рис. 6. Текущий каталог

Чтобы сменить текущий каталог, нужно ввести команду: *cd* (*про-бел*) *путь_к_папке* и нажать <Enter> (рис. 7).

```

C:\Users\Evgeny\Desktop>cd nop
C:\Users\Evgeny\Desktop\nop>

```

Рис. 7. Выполнение команды *cd*

Если требуется открыть новый каталог на том же диске, где находится текущий, то диск прописывать не обязательно, достаточно указать имя нужного каталога.

Для того чтобы с помощью команды *cd* перейти на другой диск, необходимо добавить параметр (иногда его называют флагом) */D*, т.е. синтаксис команды для перехода на другой диск будет выглядеть как *cd /D D:* (рис. 8).

```

C:\>cd /D D:\
D:\>

```

Рис. 8. Выполнение команды *cd /D D:*

Команда *mkdir*. Позволяет создавать новые папки или подпапки заданного названия. Синтаксис команды *mkdir (пробел) имя_папки* (рис. 9).

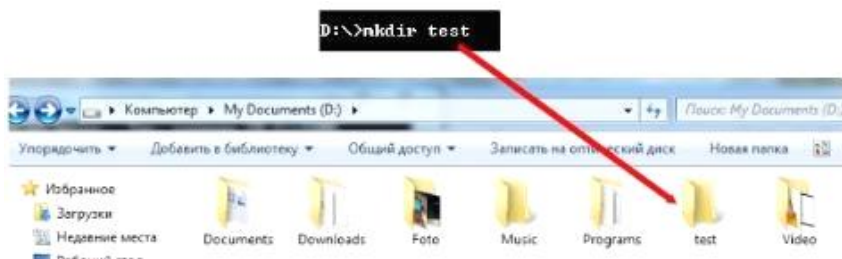


Рис. 9. Создание папки *test* с помощью команды *mkdir*

По умолчанию команда *mkdir* создает папки в текущем каталоге. Если нужно создать папку в другом каталоге, требуется дополнительно указать путь к нему (рис. 10).

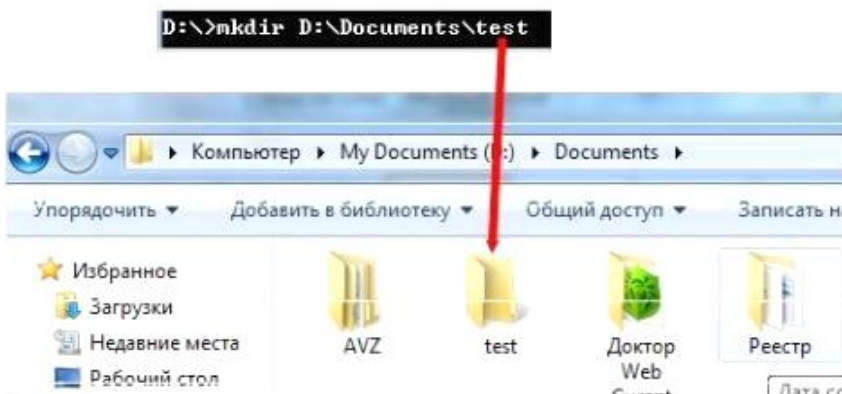


Рис. 10. Выполнение команды *mkdir* с указанием каталога

С помощью этой же команды можно создать сразу дерево каталогов (рис. 11).

Команда *mkdir* не может быть выполнена, если каталог или файл с заданным именем уже существует.

При использовании интегрированных средств разработки (англ. Integrated Development Environment – IDE) создание исполняемого файла из исходного текста происходит после нажатия одной кнопки;

при этом результат каждого отдельного этапа, как правило, скрыт от пользователя IDE.

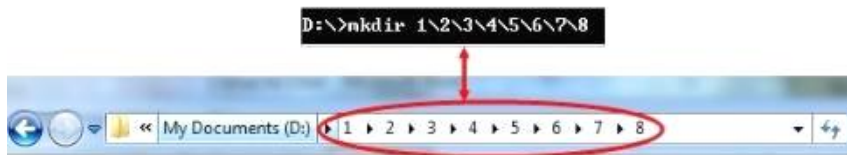


Рис. 11. Создание дерева каталогов

Рассмотрим данный процесс поэтапно при помощи набора компиляторов GCC, а также другие его возможности.

Дальнейшее описание предполагает работу в операционной системе семейства Windows с установленным IDE Dev-Cpp. Будет использован набор компиляторов GCC, входящий в состав IDE Dev-Cpp. Путь установки предполагается следующий: *C:\dev-cpp*, рабочий каталог для хранения исходных текстов – *c:\users*. При использовании других путей к рабочему каталогу и Dev-Cpp необходимо соответствующим образом отредактировать приведенные команды.

При использовании операционной системы из семейства Linux не требуется устанавливать IDE Dev-Cpp и указывать путь к программам, входящим в состав GCC. При использовании языка программирования C++ вместо *gcc* следует использовать *g++*.

GCC – GNU Compiler Collection – набор компиляторов и сопутствующих утилит, разработанный в рамках движения GNU.

Порядок выполнения работы

Изучение этапов компиляции при помощи набора компиляторов GCC

Для начала создадим простой файл исходного текста программы на языке C с использованием любого текстового редактора (например, Notepad++) и сохраним его под именем *hworld.c* в рабочем каталоге:

```
#include <stdio.h>
int main()
{
    printf("Hello World\n");
    return 0;
}
```

После этого можно командой `C:\dev-cpp\bin\gcc -o c:\users\hworld.exe c:\users\hworld.c` создать исполняемый файл, однако в данной работе рассмотрим каждый этап отдельно. В дальнейшем полный путь к файлам в командах будет опускаться; предполагается, что все исходные файлы расположены в рабочем каталоге `c:\users`. При использовании новой программы из комплекта GCC путь к ней будет указан лишь в первый раз.

Применив команду `gcc -E -o hworld.i hworld.c`, создадим файл `hworld.i`, содержащий исходный текст, обработанный препроцессором. В данный файл будет добавлено содержимое заголовочных файлов, удалены комментарии и раскрыты макросы.

Применив команду `gcc -S -o hworld.s hworld.c`, получим файл с ассемблерным кодом, соответствующим исходному тексту.

Результатом команды `gcc -c -o hworld.o hworld.c` будет объектный файл, содержащий блоки готового к исполнению машинного кода, блоки данных, а также список определенных в файле функций и внешних переменных. Но при этом в нем не заданы абсолютные адреса ссылок на функции и данные. Объектный файл не может быть запущен на исполнение непосредственно, но в дальнейшем может быть объединен с другими объектными файлами.

Важно также не менять расширения созданных файлов, так как они указывают GCC, какие этапы компиляции уже были пройдены.

Оптимизация кода

GCC позволяет провести оптимизацию кода во время компиляции. Количество оптимизирующих преобразований компилятора велико, вследствие чего вводится понятие «уровня оптимизации» как множества используемых оптимизирующих преобразований. Как правило, компиляторы имеют несколько уровней оптимизации.

Процесс оптимизации

Создадим следующий файл под именем `optim.c`:

```
#include <stdio.h>
int main()
{
    int i;
```

```

while (i<=10)
i++;
printf("%d",i);
return 0;
}

```

Теперь выполним команды:

```

gcc -S optim.c
gcc -S -O1 -o optim1.s optim.c
gcc -S -O2 -o optim2.s optim.c
gcc -S -O3 -o optim3.s optim.c

```

После этого в файле *optim.s* будет ассемблерный код без оптимизаций, в файлах *optim1.s*, *optim2.s* и *optim3.s* – код с различной степенью оптимизации.

Раздельная компиляция и статические библиотеки

GCC поддерживает раздельную компиляцию и не требует специальных указаний от пользователя.

Раздельная компиляция файлов обычно делится на несколько этапов:

- скомпилировать каждый исходный файл (.c) в объектный файл (.o);
- связать все объектные файлы в исполняемый файл.

Исходные файлы независимы, поэтому нужны **заголовочные файлы**, чтобы иметь возможность предоставлять «информацию» (объявления) о функциях в данном модуле, чтобы позволить любому другому модулю использовать их.

Далее будет принято, что файл *main.c* содержит исходный текст основной программы, файлы *first.c*, *second.c* – исходные тексты используемых в основной программе внешних функций, а файлы *first.h* и *second.h* – соответствующие им заголовочные файлы, подключенные в файле *main.c*.

Для начала с помощью команды `gcc -c` создадим из исходных файлов объектные. Компилировать можно все файлы одновременно одной командой, перечисляя составные файлы через пробел после ключа `-c` или каждый файл в отдельности.

Затем, применив команду `gcc -o main.exe main.o first.o second.o`, можно объединить три ранее созданных объектных файла в один исполняемый файл *main.exe*.

Если внести изменения в какую-то из процедур, то компилировать заново все файлы не придется, понадобится лишь скомпилировать измененный файл и собрать результирующий файл.

Создание статической библиотеки

Для создания статической библиотеки потребуется заголовочный файл *hworld.h*, содержащий прототипы функций из *first.c* и *second.c*. Данный заголовочный файл будет подключен в файле *main.c*.

Выполним последовательность команд:

```
gcc -cfirst.c
gcc -c second.c
C:\dev-cpp\bin\ar crs libhworld.a first.o second.o
```

Результатом выполнения программы *ar* будет создание библиотеки *libhworld.a*, содержащей объектные файлы *first.o* и *second.o*. Для программы *ar* заданы следующие опции:

c *arname* – создать архив; если архив с именем *arname* не существует, он будет создан, в противном случае файлы будут добавлены к имеющемуся архиву;

r – задает режим обновления архива; если в архиве файл с указанным именем уже существует, он будет удален, а новый файл дописан в конец архива;

s – добавляет (обновляет) индекс архива, необходимый для ускорения работы с библиотекой. Чтобы найти нужное определение, не нужно просматривать таблицы символов всех файлов архива, можно сразу перейти к файлу, содержащему искомое имя.

Если файл библиотеки уже существует, то архиватор будет анализировать содержимое архива, добавит новые объектные файлы и заменит старые обновленными версиями.

Для компиляции с использованием библиотеки используется команда

```
gcc -o main main.o -L. -lhworld
```

Используемые библиотеки перечисляются через ключ *-l*, после которого указывается название библиотеки без префикса *lib* и окончания *.a*. Опция *-lname* позволяет подключить библиотеку с именем *libname*; в описанном случае в качестве *name* указано *hworld*, такое же имя должен иметь подключаемый заголовочный файл.

Опция *-L <путь>* передает путь к библиотеке, в описанном случае *-L.* показывает, что искать следует в каталоге с исходным текстом.

Задания к лабораторной работе

Часть I

1. Написать программу 1 в соответствии с вариантом при помощи любого текстового редактора.

2. Провести поэтапную компиляцию исходного текста написанной программы, разобраться в результатах, полученных на каждом этапе компиляции.

3. Провести оптимизацию кода написанной программы с помощью набора компиляторов GCC, пояснить внесенные для оптимизации кода изменения.

Часть II

1. Написать программы 2 и 3 в соответствии с вариантом при помощи любого текстового редактора. Функции для работы с массивом вынести в отдельные файлы: в одном файле – для ввода/вывода массива, в другом – для обработки массива. В обеих программах должны использоваться одни и те же функции для ввода/вывода массивов, описанные в одном из этих файлов.

2. Провести раздельную компиляцию написанных файлов.

3. Скомпилировать обе программы, используя созданные объектные файлы; обе программы должны использовать один и тот же объектный файл с функциями для ввода/вывода массива.

4. Создать статическую библиотеку для ввода/вывода и обработки массива и продемонстрировать возможности по ее подключению.

Варианты задания

Вариант 1

1. Дано натуральное n . Вычислить n сомножителей произведения

$$\frac{2}{1} * \frac{2}{3} * \frac{4}{3} * \frac{4}{5} * \frac{6}{5} * \frac{6}{7} * \dots$$

2. Сформировать новый массив из элементов массива M (25), встречающихся в этом массиве только один раз.

3. Определить, представляют ли собой элементы массива A (20) возрастающую последовательность.

Вариант 2

1. Определить количество натуральных трехзначных чисел, сумма цифр которых равна заданному числу N .

2. Записать элементы массива C (20) в обратном порядке $\{C_{20}; C_{19}; C_{18}; \dots; C_2; C_1\}$. Вспомогательный массив не использовать.

3. Вставить число 100 после второго положительного элемента массива A (15).

Вариант 3

1. Дано натуральное число N . Составить программу для сравнения цифр старшего и младшего разрядов этого числа.

2. Определить количество элементов массива M (22), больших среднего арифметического значения элементов этого массива.

3. В массиве A (45) найти локальные максимумы, определить их местоположение. Локальным максимумом назовем элемент массива, значение которого больше, чем значения двух соседних с ним элементов.

Вариант 4

1. Дано натуральное число N . Составить программу для определения количества цифр в этом числе.

2. Найти два первых элемента в массиве C (17), значения которых не попадают в заданный с клавиатуры диапазон $[A, B]$. Поменять их местами.

3. В массиве A (35) найти минимум, определить его местоположение (с учетом возможного повторения).

Вариант 5

1. Составить программу поиска двузначных чисел, таких что если к сумме цифр этого числа прибавить квадрат этой суммы, то получится это число.

2. Удалить из массива M (25) все элементы, значения которых в этом массиве повторяются, оставив по одному.

3. Вычислить сумму отрицательных элементов массива D (19), кратных четырем.

Вариант 6

1. Найти сумму целых положительных чисел из промежутка от A до B , кратных четырем. Значения A и B вводятся с клавиатуры.

2. Поменять местами максимальный отрицательный и первый положительный элементы массива B (18).

3. Вставить число 0 в середину массива M (20), предварительно сдвинув вправо значения элементов массива, начиная с 11-го.

Вариант 7

1. Для натурального числа N получить все его натуральные делители.

2. Вычислить сумму элементов массива M (15), значения которых лежат в введенном с клавиатуры диапазоне $[X, Y]$.

3. Удалить из массива A (20) первый отрицательный элемент.

Вариант 8

1. Сумма цифр трехзначного числа кратна семи, само число так же делится на семь. Найти все такие числа.
2. Поменять местами максимальный и последний отрицательный элементы массива A (40).
3. Сформировать новый массив из элементов заданного целочисленного массива M (50), кратных семи или содержащих в записи числа цифру 7.

Вариант 9

1. Дано натуральное N . Составить программу для поиска первой цифры этого числа.
2. Удалить из массива M (26) первый положительный элемент.
3. Заменить все четные элементы массива A (20) на их квадраты, а нечетные удвоить.

Вариант 10

1. Среди четырехзначных чисел выбрать те, у которых все четыре цифры различны.
2. Удалить из массива B (50) все элементы, кратные трем или пяти.
3. Определить, есть ли в массиве Q (10) заданное число X , и если нет, то найти ближайшее к нему.

Вариант 11

1. Поменять местами цифры старшего и младшего разрядов данного натурального числа (например, из числа 3879 получится 9873).
2. Найти сумму элементов массива A (45), находящихся между максимальным и минимальным значениями.
3. Сформировать новый массив из положительных нечетных элементов заданного массива P (20).

Вариант 12

1. Ввести натуральные числа A и B . Определить все числа, кратные A и B , меньшие $A*B$.
2. Определить местоположение элементов массива A (30), не встречающихся в массиве B (15).
3. Найти сумму четных элементов массива M (15), имеющих четные индексы.

Вариант 13

1. Определить, являются ли натуральные числа A и B взаимно простыми. Взаимно простые числа не имеют общих делителей, кроме единицы.

2. Определить, есть ли в массиве M (15) пары соседних одинаковых элементов.

3. Найти наибольший отрицательный элемент массива A (23) и удалить его.

Вариант 14

1. Дано натуральное число N . Вычислить $S=1+2^2+3^3+\dots+N^N$. Формулу возведения в степень не использовать.

2. Поменять местами первый и последний отрицательные элементы массива B (18).

3. Определить, является ли массив M (20) перестановкой последовательности натуральных чисел от 1 до 20, т. е. проверить, все ли числа из этого диапазона входят в указанный массив.

Вариант 15

1. Исходное данное – натуральное число K , выражающее площадь. Написать программу для нахождения всех таких прямоугольников, площадь которых равна K и стороны выражены натуральными числами.

2. Дан массив целых чисел B (30). Определить, сколько из них делится на семь без остатка.

3. Вычислить $Z = \frac{S_n + S_0}{S_n - S_0}$, где S_n и S_0 – суммы положительных и отрицательных элементов массива A (70).

Вариант 16

1. Определить количество натуральных трехзначных чисел, сумма цифр которых равна заданному числу N .

2. Поменять местами первый и последний отрицательные элементы массива B (20).

3. Вставить число 10 после второго отрицательного элемента массива A (15).

Вариант 17

1. Ввести натуральные числа A и B . Определить все числа, кратные A и B , меньшие $A*B$.

2. Определить местоположение элемента массива A (30), который меньше суммы элементов массива в три раза.

3. Найти сумму четных элементов массива M (15), имеющих четные индексы.

Вариант 18

1. Определить, являются ли натуральные числа A и B взаимно простыми. Взаимно простые числа не имеют общих делителей, кроме единицы. Значения A и B задавать случайным образом.

2. Определить, есть ли в массиве M (20) пары соседних одинаковых элементов.

3. Найти наибольший положительный элемент массива A (23) и вставить его на первое место в массиве.

Контрольные вопросы

1. Что такое консоль и для чего она нужна?
2. Какие команды используются в консоли?
3. Что такое GCC и что входит в его состав?
4. Что такое компиляция?
5. Этапы компиляции.
6. Параметры программы gcc.
7. Что происходит при оптимизации кода?
8. Что такое раздельная компиляция?
9. Как создать и использовать статические библиотеки?

ЛАБОРАТОРНАЯ РАБОТА № 2

ОСОБЕННОСТИ ИСПОЛЬЗОВАНИЯ ОТЛАДЧИКА GDB

Важным этапом разработки программного обеспечения является отладка написанной программы. Рассмотрим данный процесс при использовании программы GDB (GNU Debugger), входящей в набор утилит GCC.

Отладчик – основное инструментальное средство для отладки программ. Он служит для поиска ошибок самого разного вида путем прогона программы в особом, отладочном, режиме. Отладочный режим отличается от обычного тем, что в нем выполнение программы можно приостанавливать и продолжать в любом месте и в любой момент, выполнять программу по шагам, смотреть состояние ячейки памяти или регистра.

Дальнейшее описание предполагает работу в операционной системе семейства Windows с установленным IDE Dev-Cpp. Будет использован набор компиляторов и утилит GCC, входящий в состав IDE Dev-Cpp. Путь установки для Dev-Cpp предполагается следующий: `C:\dev-cpp`. Рабочий каталог для хранения исходных текстов – `c:\users`. При использовании других путей к рабочему каталогу и Dev-Cpp необходимо соответствующим образом отредактировать приведенные команды.

При использовании операционной системы из семейства Linux не требуется устанавливать IDE Dev-Cpp и указывать путь к программам, входящим в состав GCC.

Порядок выполнения работы

Подготовка исходного текста программы к использованию отладчика

Для демонстрации процесса отладки создадим файл *gdbtest.c* со следующим исходным текстом программы:

```
#include <stdio.h>
int main()
{
    int i=0;
    for(i=1;i<5;i++);
    {
        printf("2i=%d\n",2*i);
    }
    return 0;
}
```

Предполагаемым результатом работы данной программы должен быть вывод на экран чисел 2,4,6,8, однако из-за допущенной ошибки на экран выводится только число 10.

Теперь скомпилируем этот файл командой *C:\dev-cpp\bin\gcc -g c:\users\gdbtest.c -o c:\users\gdbtest.exe*.

Параметр *-g* отвечает за добавление в исполняемый файл отладочной информации.

Использование отладчика

Следующим шагом будет запуск отладки программы командой *C:\dev-cpp\bin\gdb c:\users\gdbtest.exe*. После ее выполнения появится информация об отладчике и приглашение к вводу команд (*gdb*). После приглашения можно вводить различные команды. Многие из них можно сокращать до одной-двух букв.

Можно использовать клавишу <Tab> для завершения имени команды, переменной или функции. Простое нажатие клавиши <Enter> без указания команды повторяет последнюю ранее введенную команду.

Для завершения работы GDB введите команду *quit* (сокращается до *q*). Для начала проверим работоспособность программы командой *run*. Результат работы программы будет выведен на экран.

Теперь попробуем пошаговое выполнение программы. Для начала командой *break 4* или *b 4* установим точку останова выполнения на четвертой строчке исходного текста программы. После выполнения

команды *run* или *r* программа остановится и выведет на экран текущую строчку исходного кода (в данном случае `for(i=1;i<5;i++);`).

Теперь можно проверить значение переменной *i* с помощью команды *print i* или *p i*. После этого командой *step* или *s* перейдем на следующую строчку исходного текста и вновь проверим значение переменной *i*. Очевидно, что ее значение отличается от ожидаемого (5 вместо 1), что должно подсказать строчку, в которой допущена ошибка. Очевидно, что в нашем случае ошибка допущена в строке с началом цикла `for` – неверно установленная точка с запятой привела к отсутствию тела цикла. Исправим эту ошибку и заново откомпилируем программу. Предварительно следует завершить работу отладчика командой *quit* или *q*.

Вновь запустим отладчик для исправленной версии программы, и установим точку останова на четвертой строке исходного текста. Вместо того чтобы на каждом шаге работы программы проверять значение переменной *i*, добавим ее в наблюдаемые командой *display i*. Теперь, используя команду *step* или *continue*, пройдем до конца выполнения программы, проверяя соответствие значения переменной *i* ожидаемому.

Основные команды отладчика

help (h) – вывести справку по командам, при использовании без параметра вывести везде список разделов справки, информация по каждому разделу выводится командой *help<имя раздела>*;

run (r) – начать исполнение программы до первой точки останова или возникновения ошибки вроде `Floating point exception` или `Segmentation fault`. Для перенаправления ввода-вывода команды можно использовать команду *run<input_file>output_file*;

continue (c) – продолжить выполнение программы до следующей точки останова или ошибки;

step (s) – продолжить выполнение программы пока управление не достигнет следующей строки исходного текста. При этом происходит вход в функцию, т.е. при вызове каждой функции программа также останавливается;

next (n) – работает аналогично команде *step*, но при этом функции вызываются без остановки;

finish – продолжить выполнение программы до возврата из текущей функции, напечатать возвращаемое значение;

print <expr> (p) – отобразить значение выражения *expr*. Допускается использование практически любого выражения языка C/C++, в том числе и вызовы функций. Например, *p a, p*

sqrt(n), p (a&b)>>3 – если имеется динамический массив (например, `int * p=new int[10]`), то при выводе его на экран (`print p`) будет напечатано значение указателя, т.е. адрес в памяти компьютера. Если хочется вывести весь массив (или его часть), то необходимо воспользоваться оператором создания виртуального массива **@**, левый операнд которого – первый элемент запрашиваемого массива, а правый – длина желаемого массива. Например, `print *p@10` или `printp[0]@10` ;

display <expr> – добавить выражение *expr* к списку автоматического отображения. Каждое такое выражение получает свой номер;

delete display n – удалить выражение с номером *n* из списка автоматического отображения;

disable display n – отключить отображение элемента с номером *n* из списка, не удаляя его;

enable display n – включить отображение элемента *n* из списка, отключенного ранее;

display – напечатать значения всех автоматически отображаемых выражений, как это происходит при остановке программы;

info display – вывести список автоматически отображаемых выражений. В произвольном месте программы можно поставить точку останова, по достижении которой команда остановится. При этом можно исследовать значения переменных, продолжить выполнение программы при помощи команд *s*, *n* или выполнить другие команды GDB.

break n (b) – установить точку останова на строке исходного текста с номером *n*. Все точки останова имеют свои номера, определяемые при их установке, если *n* не указана, точка останова устанавливается на текущей строке исходного текста;

break <function> – установить точку останова на функции **function**. Исполнение программы остановится при вызове этой функции. Распространенный пример – *breakmain* для того, чтобы можно было пошагово исполнять программу с самого начала;

tbreak... – установка разовой точки останова. Работает аналогично *break*, но точка останова автоматически удаляется при достижении ее;

break <n or function> if <cond> – установка условной точки останова. Выполнение останавливается на указанной строке или функции, если выполнено условие *cond*. Пример: *break120 if a==15*;

clear <function> или **clear n** – удалить точку останова на функции *function* или в строке с номером *n*;

delete <диапазон> (**d**) – удалить точки останова с номерами из указанного диапазона. Например, *delete 1-5* или *delete 4*. Если диапазон не задан, удаляются все точки останова;

disable <диапазон> – отключить точки из заданного диапазона;

enable <диапазон> – включить точки из заданного диапазона;

enable once <диапазон> – точки из заданного диапазона включаются до первого срабатывания, после чего отключаются;

enable delete <диапазон> – точки из заданного диапазона включаются до первого срабатывания, после чего удаляются;

condition n <cond> – превратить точку останова с номером *n* в условную с условием *cond*;

condition n – превратить точку останова с номером *n* в безусловную;

info break – вывести информацию обо всех имеющихся точках останова;

watch <expr> – установить точку наблюдения на выражении <expr>. Исполнение программы останавливается, когда значение **expr** сохраняется программой и его величина изменяется. Пример: *watch a*;

rwatch <expr> – установить точку наблюдения, которая остановит программу, когда выражение *expr* считывается программой;

awatch <expr> – установить точку наблюдения, которая остановит программу, если выражение *expr* считывается или сохраняется;

info watch – вывести информацию о всех точках наблюдения; Для вывода исходного текста программы используется команда *list (l)* в различных вариантах:

list n – вывести строки исходного кода, вокруг строки с номером *n*;

list <function> – вывести начало кода функции *function*;

list n,m – вывести строки с номерами от *n* до *m*;

list n, – вывести строки, начиная с *n*;

list, m – вывести строки до номера *m*;

list – вывести еще немного строк. По умолчанию выводится 10 следующих строк, после ранее выведенных. Это значение можно изменить командой *set listsize <число>*; Всякий раз, когда вызывается функция, информация о вызове сохраняется в стеке. Информация сохраняется в блоке данных, называемом кадром стека.

where – вывести содержимое стека;

frame n m – переключиться в кадр с номером *n*. При этом можно просматривать значения локальных переменных в контексте этого кадра;

up n – переместиться по стеку вверх на *n* кадров. По умолчанию *n* равно 1;

down n – переместиться по стеку вниз на *n* кадров;

info frame – вывести информацию о текущем кадре;

info args – вывести информацию об аргументах текущего кадра;

info locals – вывести информацию о всех локальных переменных текущего кадра.

Задания к лабораторной работе

Часть I

1. Написать программу 1 в соответствии с вариантом при помощи любого текстового редактора.

2. Скомпилировать программу с добавлением в файл отладочной информации.

3. Используя отладчик GDB, проверить значение вычисляемого в цикле выражения на каждом шаге цикла. При использовании нескольких циклов проверить все значения.

4. В отчете привести используемые команды отладчика и полученный результат.

Часть II

1. Написать программу 2 в соответствии с вариантом при помощи любого текстового редактора.

2. Для ввода и вывода строки использовать отдельные функции.

3. Скомпилировать программу с добавлением в файл отладочной информации.

4. Используя отладчик GDB, проверить содержимое стека при входе в функции ввода и вывода строки и выходе из них.

5. В отчете привести содержимое стека и используемые команды.

Часть III

1. Написать программу 3 в соответствии с вариантом при помощи любого текстового редактора.

2. Для ввода и вывода строки использовать отдельные функции, помещенные в статическую библиотеку.

3. Скомпилировать программу с добавлением в файл отладочной информации.

4. Используя отладчик GDB, проверить содержимое стека при входе в функции ввода и вывода строки и выходе из них.
5. В отчете привести содержимое стека и используемые команды.

Варианты задания

Вариант 1

1. M и N – числитель и знаменатель обыкновенной дроби. Составить программу, позволяющую сократить эту дробь.
2. Дана строка символов до точки. Выделить в ней все русские буквы, сделав их заглавными.
3. Дана строка символов до точки. Определить, является ли она палиндромом. (Палиндром слева направо и справа налево читается одинаково, например: «Леша на полке клопа нашел».) Пробелы и знаки препинания, а также регистр букв не учитываются.

Вариант 2

1. Представить натуральное число N в виде произведения простых сомножителей.
2. Дана строка символов. Удалить из нее все слова нечетной длины. Слова отделяются друг от друга одним пробелом.
3. Дана строка символов. Подсчитать, сколько раз в ней встречается подслово «АВВА».

Вариант 3

1. Составить программу для определения, является ли натуральное число k степенью числа.
2. Дана строка символов (введена с клавиатуры), состоящая из цифр от 0 до 9. Составить новую строку из букв от A (соответствует цифре 0) до J (соответствует цифре 9).
3. Дана символьная строка. Заменить в ней все буквы «О» на «Ъ», а буквы «Е» – на случайные символы.

Вариант 4

1. Дано целое $m > 1$. Получить наибольшее целое k , при котором $4k < m$.
2. Дана строка символов до точки. Определить, входят ли в состав заданной строки цифры. Сформировать из них новую строку.
3. Дана строка символов до точки. Изменить ее таким образом, чтобы все символы отделялись друг от друга одним пробелом.

Вариант 5

1. Найти наименьший общий делитель трех натуральных чисел (единица будет считаться наименьшим общим делителем только в том случае, когда других общих делителей у заданных чисел нет).

2. Дана строка символов до точки. Записать слова этой строки в обратном порядке (мама мыла раму → раму мыла мама).

3. Дана строка символов до точки. Подсчитать, сколько и каких знаков препинания она содержит.

Вариант 6

1. Определить, является ли данное натуральное число N факториалом какого-нибудь числа, если «да», то какого.

2. Дана строка символов. Определить, является ли она правильным скобочным выражением.

3. Дана символьная строка. Проверить, все ли слова после точки начинаются с заглавной буквы. Если нет – исправить.

Вариант 7

1. Найти число Фибоначчи, ближайшее к заданному натуральному числу N .

2. Даны две строки. Составить третью строку из слов, имеющих в обеих данных строках.

3. Дана строка символов. Выделить и вывести слова, ограниченные пробелом или знаками препинания: запятая, точка, двоеточие, точка с запятой.

Вариант 8

1. С клавиатуры вводится последовательность чисел, признак окончания ввода – ввод нуля. Найти максимальное из них.

2. Дана строка символов. Изменить строку – во всех словах, имеющих нечетное количество символов, средний символ удалить.

3. Дана строка символов. Найти в строке самое длинное слово и вывести его на экран.

Вариант 9

1. С клавиатуры вводится последовательность натуральных чисел, признак окончания ввода – ввод нуля. Найти все числа, оканчивающиеся на 7.

2. Дана строка символов до точки. Оставить в ней только слова, начинающиеся на буквы «А», «D», «K», «P».

3. Дана строка символов до точки. Удалить из нее все сочетания «ас».

Вариант 10

1. Составить программу для определения, в каких двузначных числах удвоенная сумма цифр равна их произведению.

2. Дана символьная строка. Оставить в ней только слова, содержащие хотя бы одну букву «А».

3. Дана строка, состоящая из букв и цифр. Преобразовать ее так, чтобы сначала шли буквы, а потом – все цифры исходной строки.

Вариант 11

1. С клавиатуры вводится число N . Определить, может ли оно быть восьмеричным (т.е. состоять только из цифр меньше 8).
2. Дана строка символов, состоящая из букв и цифр. Найти количество букв в данной строке.
3. Дана строка символов. Поменять местами первое и последнее слова.

Вариант 12

1. Дано натуральное число $n > 10$. Составить программу для вычисления значения
$$y = n * \cos x + (n - 1) * \cos 2x + (n - 2) * \cos 3x + \dots + 2 * \cos(n - 1)x + \cos nx.$$
2. Дана строка символов до точки. Удалить из нее все слова, начинающиеся с буквы «а».
3. Дана строка символов до точки. Удалить из нее все повторные вхождения букв.

Вариант 13

1. Дано натуральное число N . Вычислить произведение
$$\left(1 + \frac{1}{1^2}\right) * \left(1 + \frac{1}{2^2}\right) * \dots * \left(1 + \frac{1}{N^2}\right).$$
2. Дана символьная строка. Удалить из нее все символы, не являющиеся буквами.
3. Дана строка, состоящая из букв от A до J . Составить новую строку из цифр от 0 (соответствует цифре A) до 9 (соответствует цифре J).

Вариант 14

1. Составить программу для вычисления значения
$$Y = \sin 1 + \sin 1.1 + \sin 1.2 + \dots + \sin 2.$$
2. Дана строка символов. Поменять местами N первых и N последних символов заданной строки.
3. Дана строка, состоящая из букв и цифр. Проверить, является ли данная строка представлением числа в шестнадцатеричной системе счисления.

Вариант 15

1. Даны два натуральных числа X и Y . Составить программу для вычисления суммы кубов всех четных чисел, лежащих в диапазоне $[X, Y]$.
2. Дана строка символов до точки. Инвертировать каждое слово в заданной строке.
3. Дана строка символов до точки. Определить, сколько и каких латинских букв в ней содержится.

Вариант 16

1. M и N – числитель и знаменатель обыкновенной дроби. Составить программу, позволяющую сократить эту дробь.

2. Дана строка символов до точки. Удалить из нее все слова, начинающиеся с буквы «а».

3. Дана строка символов. Поменять местами первое и последнее слова.

Вариант 17

1. Дано натуральное число $n > 10$. Составить программу для вычисления значения

$$y = n * \cos x + (n - 1) * \cos 2x + (n - 2) * \cos 3x + \dots + 2 * \cos(n - 1)x + \cos nx.$$

2. Дана строка символов до точки. Оставить в ней только слова, начинающиеся с букв «А», «D», «X», «P».

3. Дана строка символов до точки. Удалить из нее все сочетания «лас».

Вариант 18

1. Найти число Фибоначчи, ближайшее к заданному натуральному числу V .

2. Дана строка, которая заканчивается точкой. Определить количество заглавных букв.

3. Дана строка символов. Выделить и вывести слова, ограниченные пробелом или знаками препинания: запятая, точка, двоеточие, точка с запятой.

Контрольные вопросы

1. Что такое GDB?
2. Что такое отладка программного обеспечения?
3. Каким образом должны быть подготовлены программы, чтобы их можно было исследовать с помощью GDB?
4. Что такое «точка останова»? Сколько точек останова можно задать в процессе отладки программы? Можно ли задать условие срабатывания точки останова?
5. Что такое «точка наблюдения» и как ее использовать?
6. Что такое стек? Что в него заносится при работе программы?
7. Просмотр стека с помощью GDB.
8. Список автоматического отображения и работа с ним.
9. Просмотр исходного текста программы с помощью GDB.
10. Основные команды GDB.

11. Какую именно информацию необходимо добавить в программу на этапе компиляции для работы отладчика?

ЛАБОРАТОРНАЯ РАБОТА № 3

СОЗДАНИЕ ДИНАМИЧЕСКИХ БИБЛИОТЕК ПРИ ПОМОЩИ НАБОРА КОМПИЛЯТОРОВ И УТИЛИТ GCC И ИХ ПРИМЕНЕНИЕ

Динамическая библиотека — это созданная специальным образом библиотека, которая присоединяется к результирующей программе в два этапа. Первый этап — компиляция. На этом этапе линковщик встраивает в программу описания требуемых функций и переменных, которые присутствуют в библиотеке. Сами объектные файлы из библиотеки не присоединяются к программе. Присоединение этих объектных файлов осуществляет системный динамический загрузчик во время запуска программы. Загрузчик проверяет все библиотеки, связанные с программой, на наличие требуемых объектных файлов, затем загружает их в память и присоединяет в копии запущенной программы, находящейся в памяти.

Сложный процесс загрузки динамических библиотек замедляет запуск программы, но при этом если другая запускаемая программа связана с этой же загруженной динамической библиотекой, то она использует ту же копию библиотеки. Это означает, что требуется гораздо меньше памяти для запуска нескольких программ, сами загрузочные файлы меньше по размеру, что экономит место на дисках.

Создание подобных библиотек имеет свои особенности в разных операционных системах. При выполнении задания допускается использовать любой способ, но при защите лабораторной работы необходимо продемонстрировать знание обоих.

Порядок выполнения работы

Создание динамической библиотеки

Для примера создадим следующие файлы:

```
//-----fun.h
void func(void);
//-----first.c
#include <stdio.h>
#include "fun.h"
```

```

void func(void)
{
printf("First function...\n");
}
//-----second.c
#include <stdio.h>
#include "fun.h"
void func(void)
{
printf("Second function...\n");
}
//-----main.c
#include <stdio.h>
#include "fun.h"
int main(void)
{
func();
printf("Main function...\n");
return 0;
}

```

Файлы *.c необходимо скомпилировать в объектные файлы, при этом рекомендуется использовать параметр *-fPIC*, например:

```
gcc -c -o main.o -fPICmain.c
```

Опция *-fPIC* требует от компилятора при создании объектных файлов порождать позиционно-независимый код (PIC – Position Independent Code), его основное отличие в способе представления адресов. Вместо указания фиксированных (статических) позиций все адреса вычисляются исходя из смещений заданных в глобальной таблице смещений (global offset table – GOT). Формат позиционно-независимого кода позволяет подключать исполняемые модули к коду основной программы в момент ее загрузки. Соответственно основное назначение позиционно-независимого кода – создание динамических (разделяемых) библиотек.

Для ОС семейства Windows в простейшем случае создание динамических библиотек выглядит так:

```

gcc -shared -o lib1.dll first.o
gcc -shared -o lib2.dll second.o

```

Опция *-shared* – указывает gcc, что в результате должен быть собран не исполняемый файл, а разделяемый объект – динамическая библиотека. Их подключение:

```
gcc -omain1.exemain.o -L./ -llib1
gcc -o main2.exe main.o -L./ -llib2
```

Подключение динамических библиотек в процессе выполнения программы

Для начала создадим библиотеку с функциями для загрузки других библиотек:

```
// ----- load.c
#include "load.h"
#include "fun.h"
#include <stdio.h>
#include <windows.h>
void LoadRun(const char * const s) {
    void * lib;
    void (*fun)(void);
    lib = LoadLibrary(s) //загрузка библиотеки в память;
    if (!lib) {
        printf("cannot open library \"%s\\n", s);
        return;
    }
    fun = (void (*)(void))GetProcAddress((HINSTANCE)lib, "func")
//получение указателя на функцию из библиотеки;
    if (fun == NULL) {
        printf("cannot load function func\\n");
    } else {
        fun();
    }
    FreeLibrary((HINSTANCE)lib) //выгрузка библиотеки;
}
```

Здесь нужно обратить внимание на функции *LoadLibrary* и *GetProcAddress*. Первой мы загрузим в память библиотеку, переданную ей в качестве параметра. Вторая используется для получения указателя на функцию из загруженной библиотеки с указанным названием. В конце работы библиотеку необходимо выгрузить функцией *FreeLibrary*.

В данном примере мы ищем в библиотеке функцию *func*:

```
// ----- load.h
void LoadRun(const char * const s);
```

Создадим динамическую библиотеку из load.c:

```
gcc.exe -c -fPICload.c
gcc.exe -shared -o load.dll load.o
```

Теперь опишем файл основной программы. В качестве подключаемых библиотек будем использовать ранее созданные *lib1.dll* и *lib2.dll*:

```
// ----- loadmain.c
#include "load.h"
#include <stdio.h>
int main(void)
{
    int a=0,b=1;
    printf("Choose library:\n1-first.\n2-second,\n3-quit\n");
    while(b)
    {
        scanf("%d",&a);
        if(a==1)
            LoadRun("lib1.dll");
        if(a==2)
            LoadRun("lib2.dll");
        if(a==3)
            b=0;
    }
    return 0;
}
```

Скомпилируем и запустим полученную программу:

```
gcc.exe -c -fPICloadmain.c
gcc.exe -o loadmain.exe loadmain.o -L./ -lload
loadmain.exe
```

Теперь при нажатии на 1 у нас будет вызываться функция из первой библиотеке, при нажатии на 2 – из второй.

Особенности создания и использования динамических библиотек в ОС семейства Linux

В случае использования ОС семейства Linux необходимо учитывать следующие особенности:

- вместо windows.h в загрузчике использовать заголовочный файл dlfcn.h, загрузку библиотеки выполнять так: lib = dlopen(s, RTLD_LAZY);

- получение функции из библиотеки происходит так: `fun = (void (*)(void))dlsym(lib, "run")`, освобождение библиотеки – `dlclose(lib)`;
- при компиляции основного файла в конец команды добавить параметр – `ldl`(подключить стандартную библиотеку `dl`);
- для динамических библиотек использовать расширение `*.so`

Контроль версий

При разработке проекта с использованием динамических библиотек необходимо тщательно контролировать версию библиотеки во избежание проблем совместимости. В самом проекте этот контроль осуществляется различными способами в зависимости от семейства ОС. Рассмотрим эти способы.

Для ОС семейства Windows контроль версии осуществляется путем простого добавления информации о версии в имя файла и проверки его при компиляции проекта.

Для ОС семейства Linux введено понятие *soname*. Продемонстрируем его применение на примере. Для начала скомпилируем библиотеку с указанием *soname*:

```
gcc -shared -o libhello.so.2.4.0.5 -Wl,-soname,libhello.so.2 first.o second.o
```

Результатом будет файл *libhello.so.2.4.0.5*, где 2.4.0.5 – его версия, а *libhello.so* – имя. Для задания *soname* используется параметр `-Wl,-soname,libhello.so.2`. Данный параметр предназначен для непосредственного взаимодействия пользователя с линковщиком. По ходу компиляции `gcc` вызывает линковщик автоматически, автоматически же, по собственному усмотрению, `gcc` передает ему необходимые для успешного завершения задания параметры.

Если у пользователя возникает потребность самому вмешаться в процесс линковки, он может воспользоваться специальным параметром `gcc -Wl,-option,value1,value2....`, что означает передать линковщику (`-Wl`) параметр `-option`с аргументами *value1*, *value2* и т.д. В нашем случае линковщику был передан параметр *soname* с аргументом *libhello.so.2*.

Для того чтобы система, конкретно – загрузчик динамических библиотек, имела представление о том, библиотека какой версии была использована при компиляции приложения и, соответственно, необходима для его успешного функционирования, был предусмотрен

специальный идентификатор – *soname*, помещаемый как в файл самой библиотеки, так и в исполняемый файл приложения. Идентификатор *soname* – это строка, включающая имя библиотеки с префиксом *lib*, точку, расширение *so*, снова точку и одну или две (разделенные точкой) цифры версии библиотеки – *libname.so.x.y*. То есть *soname* совпадает с именем файла библиотеки вплоть до первой или второй цифры номера версии.

Пусть имя исполняемого файла нашей библиотеки *libhello.so.2.4.0.5*, тогда *soname* библиотеки может быть *libhello.so.2*. При изменении интерфейса библиотеки ее *soname* необходимо изменять. Любая модификация кода, приводящая к несовместимости с предыдущими релизами, должна сопровождаться появлением нового *soname*.

Пусть для успешного исполнения некоторого приложения необходима библиотека с именем *hello*. В системе таковая имеется, при этом имя файла библиотеки *libhello.so.2.4.0.5*, а прописанное в нем *soname* библиотеки *libhello.so.2*. На этапе компиляции приложения линковщик в соответствии с опцией *-lhello* будет искать в системе файл с именем *libhello.so*. В реальной системе *libhello.so* – это символическая ссылка на файл *libhello.so.2.4.0.5*.

Получив доступ к файлу библиотеки, линковщик считает прописанное в нем значение *soname* и наряду с прочим поместит его в исполняемый файл приложения. Когда приложение будет запущено, загрузчик динамических библиотек получит запрос на подключение библиотеки с *soname*, считанным из исполняемого файла, и попытается найти в системе библиотеку, имя файла которой совпадает с *soname*. То есть загрузчик попытается отыскать файл *libhello.so.2*. Если система настроена корректно, в ней должна присутствовать символическая ссылка *libhello.so.2* на файл *libhello.so.2.4.0.5*, загрузчик получит доступ к требуемой библиотеке и далее, не задумываясь (и ничего более не проверяя), подключит ее к приложению. Представим, что мы перенесли откомпилированное таким образом приложение в другую систему, где развернута только предыдущая версия библиотеки с *soname libhello.so.1*. Попытка запустить программу приведет к ошибке, так как в этой системе файла с именем *libhello.so.2* нет. Поэтому в ОС семейства *linux* недостаточно просто скомпилировать программу командой *gcc -omainmain.o -L. -lhello -Wl,-rpath.* Сначала необходимо создать символическую ссылку на библиотеку (в нашем случае командой *ln -s libhello.so.2.4.0.5 libhello.so*) и другую, соответствующую *soname*, командой *ln -s libhello.so.2.4.0.5 libhello.so.2*

Также командой `objdump -p main | grep NEEDED` всегда можно проверить, библиотеки с каким *soname* требуются указанной программе. Также при компиляции использовался параметр `-Wl,-rpath`, – передать линковщику параметр `-rpath` с аргументом. С помощью `-rpath` в исполняемый файл программы можно прописать дополнительные пути, по которым загрузчик разделяемых библиотек будет производить поиск библиотечных файлов. В нашем случае прописан путь – поиск файлов библиотек будет начинаться с текущего каталога.

Узнать, какие разделяемые библиотеки необходимы приложению, можно и с помощью утилиты `ldd`: `lddmain`

Задания к лабораторной работе

1. Написать программу в соответствии с вариантом.
2. Массив и матрицу заполнять случайными числами от -50 до 50.
3. Функции для работы с массивами и матрицами поместить в две отдельные динамические библиотеки.
4. При запуске программы пользователь должен увидеть меню, в котором можно выбрать, с чем будет проходить работа: с матрицей или с массивом.
5. В зависимости от выбора пользователя загружается одна или другая динамическая библиотека.
6. Библиотеки должны быть скомпилированы с учетом возможного использования в ОС семейств Linux или Windows.

Варианты задания

Вариант 1

Найти сумму наибольших из отрицательных элементов матрицы A (7x8) и массива B (76).

Вариант 2

В матрице A (7x6) и массиве B (67) заменить все отрицательные числа их квадратами.

Вариант 3

Отсортировать массив M (50) и строки матрицы A (6x7) в порядке убывания.

Вариант 4

Подсчитать общее количество отрицательных элементов в массиве D (42) и матрице A (6x9).

Вариант 5

В массиве A (76) и матрице B (5x6) заменить все числа, кратные трем, единицами.

Вариант 6

Подсчитать число элементов матрицы Q (9×11) и массива R (38), остаток от деления которых на пять равен единице.

Вариант 7

Подсчитать число элементов матрицы Q (5×7) и массива R (57), кратных трем.

Вариант 8

Определить количество элементов массива A (35) и матрицы D (5×7), кратных трем и не кратных пяти одновременно.

Вариант 9

Вычислить сумму элементов массива M (11) и матрицы S (4×11), значения которых лежат в введенном с клавиатуры диапазоне $[X, Y]$.

Вариант 10

Заменить все четные по значению элементы массива A (20) и матрицы B (5×6) на их квадраты.

Вариант 11

Найти наибольший отрицательный элемент массива A (23) и матрицы B (7×5).

Вариант 12

В массиве A (45) и каждой строке матрицы B (10×6) найти локальные максимумы, определить их местоположение. Локальным максимумом назовем элемент массива, значение которого больше, чем значения двух соседних с ним элементов.

Вариант 13

В массиве A (35) и матрице B (8×5) найти минимум, определить его местоположение (с учетом возможного повторения).

Вариант 14

В массиве K (73) и матрице R (7×10) найти число элементов, которые делятся на семь без остатка.

Вариант 15

В массиве A (35) и в каждой строке матрицы B (9×8) сменить знак максимального по модулю элемента на противоположный с учетом повторений.

Вариант 16

В матрице A (8×8) и массиве B (64) заменить все положительные числа нулями.

Вариант 17

В массиве A (35) и матрице B (8×5) найти значения экстремума, определить его местоположение (вывести индекс).

Вариант 18

Определить количество элементов массива A (35) и матрицы D (5×7), кратных трем и не кратных пяти одновременно.

Контрольные вопросы

1. Отличия динамических библиотек от статических.
2. Загрузка динамических библиотек во время выполнения программы.
3. Для чего нужны динамические библиотеки?
4. Отличия при создании и использовании динамических библиотек в ОС семейств Linux и Windows.
5. Контроль версий динамических библиотек.

ЛАБОРАТОРНАЯ РАБОТА № 4

ОСОБЕННОСТИ РАБОТЫ С ПРОГРАММОЙ ДЛЯ УПРАВЛЕНИЯ КОМПИЛЯЦИЕЙ

При разработке программ на языках C или C++ важной частью для построения приложения является семейство команд компиляции и компоновки, необходимых для получения из файлов исходного кода работающих бинарных файлов. Ввод данных команд – длительная и кропотливая работа, и большинство современных сред разработки включают в себя способ помещения их в командные файлы или базы данных, которые можно автоматически вызывать для сборки приложения.

Unix-программа *make*, родоначальник всех этих средств, была разработана специально для того, чтобы помочь C-программистам управлять данными инструкциями. Она позволяет описать зависимости между файлами проекта в одном или нескольких «*make*-файлах». Каждый *make*-файл состоит из последовательности правил, каждое из которых указывает утилите *make*, что некоторый заданный целевой файл зависит от некоторого набора исходных файлов и определяет действия в случае, если любой из файлов исходного кода является более новым, чем целевой файл. Фактически программисту не требуется описывать все зависимости, поскольку программа *make* способна установить большинство очевидных зависимостей по именам файлов и расширениям.

При работе над комплексными программами, состоящими из большого числа файлов, компиляция всех файлов может занимать длительное время и требует ввода большого числа команд. Утилита *make* автоматически определяет, какие части большой программы должны быть перекомпилированы, и выполняет необходимые для этого действия.

В настоящее время существует множество утилит для отслеживания зависимостей, но *make* – одна из самых широко распространенных, в первую очередь благодаря тому, что она включена в Unix, начиная с версии PWB/UNIX (англ. Programmer's Workbench), которая содержала инструменты для разработки программного обеспечения.

Существует несколько версий *make*, основанных на оригинальной *make* или написанных с нуля, использующих те же самые форматы файлов, базовые принципы и алгоритмы, а также содержащие некоторые улучшения и расширения. Пример – *BSD-make*, основанная на работе Адама де Бура (Adam de Boor) над версией *make*, с возможностью параллельной сборки; в той или иной форме перешла в FreeBSD, NetBSD и OpenBSD.

GNU-*make* входит в большинство дистрибутивов GNU/Linux и часто используется в сочетании с GNUbuildsystem. В дальнейшем будем рассматривать GNU-*make* в версии для ОС Windows.

Программа *make* может работать не только с компилятором *gcc*, но и с любым компилятором для любого языка программирования, способным запускаться из командной строки.

Директивы утилиты *make* служат для определения зависимостей между файлами проекта и находятся в файле по имени Makefile, расположенном в каталоге сборки.

Порядок выполнения работы

Задание правил преобразования файлов

Утилита *make* предназначена для автоматизации преобразования файлов из одной формы в другую. Правила преобразования задаются в скрипте с именем *Makefile*, который должен находиться в корне рабочей директории проекта. Сам скрипт (иначе называемый *make*-файл) состоит из набора правил, которые в свою очередь описываются:

- 1) целями (то, что данное правило делает);
- 2) реквизитами (то, что необходимо для выполнения правила и получения целей);
- 3) командами (выполняющими данные преобразования).

В общем виде синтаксис *makefile* можно представить так:

```
# комментарии начинаются со значка #  
# отступы выполняются исключительно при помощи символов  
табуляции
```

```
# каждой команде должен предшествовать отступ
<цели>: <зависимости>
<команда #I>

...
<команда #n>
```

То есть правило `make` – это ответы на три вопроса:

```
{Что делаем? (цели)} --->
{Из чего делаем? (зависимости)} --->
{Как делаем? (команды)}
```

В качестве исходных файлов для начала будем использовать те, которые использовались в качестве примеров в первой лабораторной работе. В самом простом случае программа будет состоять из одного файла – `hworld.c`. При этом `makefile` должен выглядеть так:

```
hello: hworld.c
gcc -o hello.exe hworld.c
```

Перед `gcc` необходимо указать полный путь к файлу `gcc.exe`.

Таким образом, `hello` является целью, `hworld.c` – реквизитом, а `gcc -o hello.exe hworld.c` – командой. Напомним, что перед каждой командой должен быть отступ, выполняемый клавишей табуляции `<Tab>`.

Для запуска с помощью командной строки используем команду

```
mingw32-make.exe -C <ДИРЕКТОРИЯ>
```

В качестве директории необходимо указать полный путь к папке с исходными текстами программы и файлом `makefile`. Здесь и далее используется версия `make`, входящая в состав IDE Dev-Cpp. Параметр `-C` указывает путь к директории, где находятся исходные тексты программы и файл `makefile`. В результате в указанной директории появится файл `hello.exe`, запустив который, можно увидеть результат работы программы.

Рассмотрим вариант с программой из нескольких файлов с исходным текстом: `first.c` и `second.c`. Существуют несколько вариантов `makefile` для такой программы, простейший представлен в виде

```
main: first.c second.c
C:\dev-cpp\bin\gcc -o main.exe first.c second.c
```

При этом будут скомпилированы оба файла с исходным текстом программы и создан файл `main.exe`. Данный подход обладает существенным недостатком: если программа состоит из большого числа

файлов, то при внесении изменений в один из них потребуется перекомпиляция всех для создания исполняемого файла программы.

Корректнее разделять компиляцию на этапы трансляции и линковки и запускать только требуемый этап. В этом случае *makefile* будет выглядеть так:

```
first.o: first.c
C:\dev-cpp\bin\gcc -c -o first.o first.c
second.o: second.c
C:\dev-cpp\bin\gcc -c -o second.o second.c
main: first.o second.o
C:\dev-cpp\bin\gcc -o main.exe first.o second.o
```

Для создания файла потребуется указать цель, в данном случае *main*:

```
mingw32-make.exe main -C <ДИРЕКТОРИЯ>
```

После запуска *make* попытается сразу получить цель *main*, но для ее создания необходимы файлы *first.o* и *second.o*, которых пока еще нет. Поэтому выполнение правила будет отложено и *make* станет искать правила, описывающие получение недостающих реквизитов. Как только все реквизиты будут получены, *make* вернется к выполнению отложенной цели. Отсюда следует, что *make* выполняет правила рекурсивно. При повторном вызове *make* файлы *first.o* и *second.o* будут пересозданы только в случае внесения изменений в них с соответствующими исходными текстами.

Использование дополнительных целей

Помимо целей, описывающих требуемые файлы, существует набор стандартных или фиктивных (phony) целей, среди которых можно выделить следующие:

all – является стандартной целью по умолчанию. При вызове *make* ее можно явно не указывать;

clean – очистить каталог от всех файлов, полученных в результате компиляции;

install – произвести инсталляцию, *uninstall* – деинсталляцию соответственно.

Для того чтобы *make* не искал файлы с такими именами, их следует определить в *Makefile* при помощи директивы *PHONY*. Далее показан пример *Makefile* с целями *all* и *clean*:

```
.PHONY: all clean
all: main
```

```
clean:
del *.o
first.o: first.c
C:\dev-cpp\bin\gcc -c -o first.o first.c
second.o: second.c
C:\dev-cpp\bin\gcc -c -o second.o second.c
main: first.o second.o
C:\dev-cpp\bin\gcc -o main.exe first.o second.o
```

Следует обратить особое внимание на то, что если файл *main* уже имеется (остался после предыдущей компиляции) и его реквизиты не были изменены, то команда *make* ничего не станет пересобирать. Так, например, изменив заголовочный файл, случайно не включенный в список реквизитов, можно очень долго искать несоответствие. Поэтому, чтобы гарантированно полностью пересобрать проект, нужно предварительно очистить рабочий каталог:

```
mingw32-make.exe clean -C <ДИРЕКТОРИЯ>
mingw32-make.exe -C <ДИРЕКТОРИЯ>
```

Использование переменных

Чтобы избежать повторения текста в файле *makefile* и одновременно сделать его более гибким, можно использовать переменные. Переменные в *make* представляют собой именованные строки и определяются следующим образом:

```
<VAR_NAME> = <valstring>
```

Существует негласное правило, согласно которому следует именовать переменные в верхнем регистре, например:

```
SRC = first.c second.c
```

Так мы определили список исходных файлов. Для использования значения переменной ее следует разыменовывать при помощи конструкции $\$(\text{<VAR_NAME>})$, например, так:

```
gcc -o hello  $\$(SRC)$ 
```

Модифицируем представленный ранее *makefile* с использованием переменных:

```
SOURCES=first.c second.c
OBJECTS= $\$(SOURCES:.c=.o)$ 
```

```
EXECUTABLE=main.exe
FLAGS= -c -o
GCCPATH=C:\dev-cpp\bin\gcc
.PHONY: all clean
all: $(EXECUTABLE)
clean:
del *.o
$(OBJECTS): $(SOURCES)
$(GCCPATH) $(FLAGS) $@ $*.c
$(EXECUTABLE): $(OBJECTS)
$(GCCPATH) -o $(EXECUTABLE) $(OBJECTS)
```

Переменные SOURCES, OBJECTS и EXECUTABLE содержат имена файлов с исходным текстом программы, объектных файлов и исполняемого файла соответственно.

Переменная OBJECTS заполняется путем взятия имен файлов из переменной SOURCES и заменой расширения .c на .o. Переменная FLAGS хранит требуемые параметры компилятора. Переменная GCCPATH содержит полный путь к набору компиляторов gcc.

При разыменовании используются автоматические переменные \$@ и \$*. Переменная \$@ представляет собой текущую цель при использовании списка целей, переменная \$* – текущую цель без расширения.

Задания к лабораторной работе

Написать *makefile* для всех программ, написанных в предыдущих лабораторных работах с возможностью выбора требуемой в зависимости от введенной цели. Учесть, что файлы с исходным текстом программы будут лежать в подкаталогах, в которые также должны быть помещены полученные объектные и исполняемые файлы.

Контрольные вопросы

1. Что такое *make* и *makefile*?
2. Какие команды допустимы в *makefile*?
3. Какие параметры могут быть у программы *make*?
4. Как работать с переменными в *makefile*?
5. Может ли *makefile* иметь имя, отличное от *makefile*? Если да, то как с ним работать в этом случае.
6. В каком порядке выполняются цели в *makefile*?

ОСОБЕННОСТИ ИСПОЛЬЗОВАНИЯ СИСТЕМ КОНТРОЛЯ ВЕРСИЙ

Система контроля версий (СКВ) – это система, записывающая изменения в файл или набор файлов в течение времени и позволяющая вернуться позже к определенной версии.

СКВ позволяет вернуть файлы к состоянию, в котором они были до изменений, вернуть проект к исходному состоянию, увидеть изменения, увидеть, кто последний менял что-то и вызвал проблему, кто поставил задачу и когда, и многое другое.

Локальные системы контроля

Самая простая система контроля версий представляет собой простое копирование файлов в другой каталог (как правило, добавляя текущую дату к названию каталога). Такой подход очень распространен, но чаще других дает сбои. Велика вероятность ошибиться каталогом, а также сложно найти конкретную версию среди сохраненных.

Чтобы решить эту проблему, были разработаны локальные СКВ с простой базой данных, в которой хранятся все изменения нужных файлов (рис. 12).

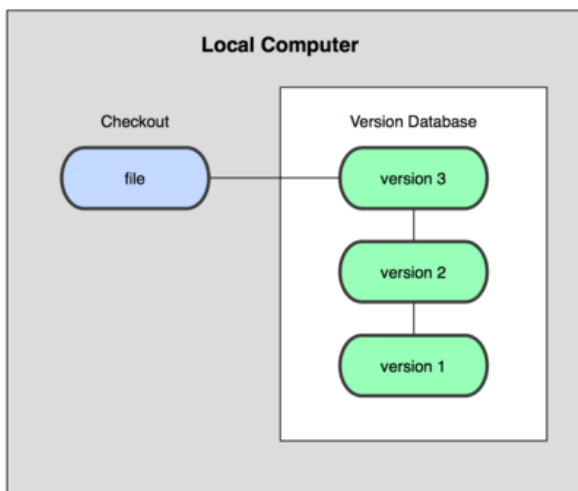


Рис. 12. Схема локальной СКВ

Одной из наиболее популярных СКВ такого типа является *rcs*. Эта утилита основана на работе с наборами патчей между парами версий (патч – файл, описывающий различие между файлами), которые хранятся в специальном формате на диске. Это позволяет пересоздать любой файл в любой момент времени, последовательно накладывая патчи. Одним из важных достоинств данной утилиты является значительное уменьшение занимаемого старыми версиями места на диске. Однако в подобных системах крайне ограничены возможности совместной работы нескольких разработчиков над одним проектом.

Централизованные системы контроля версий

Для обеспечения совместной работы нескольких разработчиков над одним проектом были созданы централизованные системы контроля версий (ЦСКВ). В таких системах (например, CVS, Subversion и Perforce) есть центральный сервер, на котором хранятся все файлы под версионным контролем, и ряд клиентов, которые получают копии файлов из него (рис. 13).

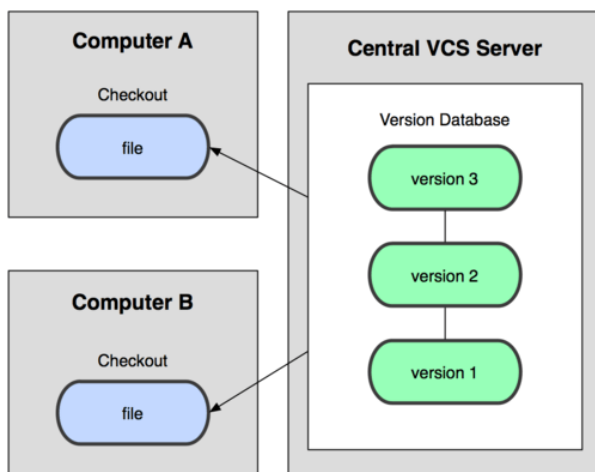


Рис. 13. Схема централизованного контроля версий

Такой подход имеет множество преимуществ над локальными СКВ: все знают, кто и чем занимается в проекте, у администраторов есть четкий контроль над тем, кто и что может делать, и возможности по администрированию ЦСКВ намного богаче, чем в случае использования локальных баз на каждом клиенте.

Однако в таком подходе есть и серьезные недостатки. Наиболее очевидный в том, что централизованный сервер является уязвимым местом всей системы. Если сервер выключается на час, то в течение часа разработчики не могут взаимодействовать и никто не может сохранить новую версию своей работы. Если повреждается диск с центральной базой данных и нет резервной копии, теряется вся история проекта, за исключением нескольких рабочих версий, сохранившихся на рабочих машинах пользователей. То же относится и к локальным системам контроля версий: если вся история проекта хранится в одном месте, можно потерять всё.

Распределенные системы контроля версий

Для устранения уязвимого звена в виде центрального сервера были созданы распределенные системы контроля версий (РСКВ). В таких системах, как Git, Mercurial, Bazaar или Darcs клиенты не просто выгружают последние версии файлов, а полностью копируют весь репозиторий. Поэтому в случае, когда «умирает» сервер, через который шла работа, любой клиентский репозиторий может быть скопирован обратно на сервер, чтобы восстановить базу данных. Каждый раз, когда клиент забирает свежую версию файлов, он создает себе полную копию всех данных (рис. 14).

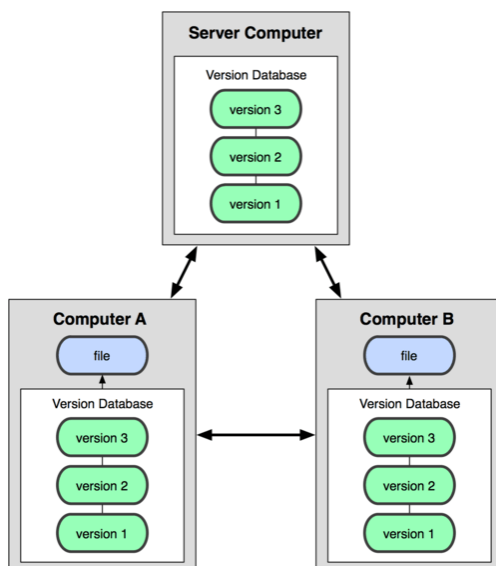


Рис. 14. Схема распределенной системы контроля версий

Кроме того, в большей части этих систем можно работать с несколькими удаленными репозиториями. Таким образом, можно одновременно работать по-разному с разными группами людей в рамках одного проекта. Так, в одном проекте можно одновременно вести несколько типов рабочих процессов, что невозможно в централизованных системах. Однако при этом увеличивается сложность синхронизации состояний репозитория на различных клиентах, а также общая уязвимость системы к утечке данных. Также повышается нагрузка на администратора проекта при разрешении конфликтов версий в ситуациях, когда несколько разработчиков предлагают разные модификации одних и тех же файлов.

Основные команды **git**

git add – добавляет содержимое рабочей директории в индекс (staging area) для последующего коммита. По умолчанию *git commit* использует лишь этот индекс, так что вы можете использовать *git add* для сборки слепок вашего следующего коммита.

git status – показывает состояния файлов в рабочей директории и индексе: какие файлы изменены, но не добавлены в индекс; какие ожидают коммита в индексе. Вдобавок к этому выводятся подсказки о том, как изменить состояние файлов.

git diff – используется для вычисления разницы между любыми двумя Git деревьями. Это может быть разница между вашей рабочей директорией и индексом (собственно *git diff*), между индексом и последним коммитом (*git diff --staged*) или между любыми двумя коммитами (*git diff master branchB*).

git difftool – просто запускает внешнюю утилиту сравнения для показа различий в двух деревьях, на случай если вы хотите использовать что-либо отличное от встроенного просмотрщика *git diff*.

git commit – берет все данные, добавленные в индекс с помощью *git add*, и сохраняет их слепок во внутренней базе данных, а затем сдвигает указатель текущей ветки на этот слепок.

git reset – используется в основном для отмены изменений. Она изменяет указатель HEAD и, опционально, состояние индекса. Также эта команда может изменить файлы в рабочей директории при использовании параметра *--hard*, что может привести к потере наработок при неправильном использовании, так что убедитесь в серьезности своих намерений, прежде чем использовать его.

git rm – используется в Git для удаления файлов из индекса и рабочей директории. Она похожа на *git add* с тем лишь исключением, что удаляет, а не добавляет файлы для следующего коммита.

git mv – это всего лишь удобный способ переместить файл, а затем выполнить *git add* для нового файла и *git rm* для старого.

git clean – используется для удаления мусора из рабочей директории. Это могут быть результаты сборки проекта или файлы конфликтов слияний.

git branch – это своего рода «менеджер веток». Она умеет перечислять ваши ветки, создавать новые, удалять и переименовывать их.

git checkout – используется для переключения веток и выгрузки их содержимого в рабочую директорию.

git log – используется для просмотра истории коммитов, начиная с самого свежего и уходя к истокам проекта. По умолчанию, она показывает лишь историю текущей ветки, но может быть настроена на вывод истории других, даже нескольких сразу, веток. Также её можно использовать для просмотра различий между ветками на уровне коммитов.

git stash – используется для временного сохранения всех незакоммиченных изменений для очистки рабочей директории без необходимости коммитить незавершённую работу в новую ветку.

git fetch – связывается с удалённым репозиторием и забирает из него все изменения, которых у вас пока нет, и сохраняет их локально.

git push – используется для установления связи с удалённым репозиторием, вычисления локальных изменений, отсутствующих в нём, и собственно их передачи в вышеупомянутый репозиторий.

Задания к лабораторной работе

Часть I. Работа с локальным репозиторием

1. Используя систему контроля версий *git*, создать локальный репозиторий для программы, написанной в третьей работе.

2. Используя команду *commit*, поместить текущее состояние файлов с исходными текстами программы и библиотек в созданный репозиторий.

3. Создать две ветви программы в репозитории: в одной убрать из программы поддержку ОС семейства Windows, в другой ОС семейства Linux.

4. Сравнить ветви и показать найденные конфликты.

5. Просмотреть историю изменений.

6. Рассмотреть команды.

7. Восстановить изначальную версию программы из репозитория.
В отчете привести использованные команды и результаты их исполнения.

Часть II. Работа с глобальным репозиторием

1. Самостоятельно изучить различия основных систем контроля версий – CVS,SVN,Git.

2. Рассмотреть доступные глобальные репозитории для любой из систем контроля версий.

3. Описать процесс регистрации и работы с одним из рассмотренных глобальных репозиториях.

В отчете привести сравнение систем контроля версий, обзор различных глобальных репозиториях для одной из них и описание процесса работы с одним из глобальных репозиториях.

Контрольные вопросы

1. Что такое система контроля версий?
2. Что представляет собой репозиторий в системе контроля версий и что конкретно он хранит?
3. Что такое ветви версий в репозитории, что происходит при слиянии ветвей и что такое конфликты ветвей?
4. Основные команды Git.
5. Основные команды SVN.
6. Различия Git и SVN.

Библиографический список

1. *Арипова, О.В.* Основы программирования на языке Си: методические указания к лабораторным работам / О.В. Арипова, О.А. Бузюкина; Балт. гос. техн. ун-т. СПб, 2006. 53 с.

2. *Директивы* препроцессора [Электронный ресурс]. 2015. URL: [https://msdn.microsoft.com/ru-ru/library/3sxhs2ty\(v=vs.120\).aspx](https://msdn.microsoft.com/ru-ru/library/3sxhs2ty(v=vs.120).aspx). Режим доступа: свободный, дата обращения 14.12.2020.

3. О GCC, компиляции и библиотеках [Электронный ресурс]. 2010. URL: <http://pyviy.blogspot.ru/2010/12/gcc.html>. Режим доступа: свободный, дата обращения 14.12.2020.

4. *Отладка* программ при помощи GDB [Электронный ресурс]. URL: <http://server.179.ru/tasks/gdb/>. Режим доступа: свободный, дата обращения 14.12.2020.

5. Система контроля версий (cvs) – Сравниваем: Git, SVN, Mercurial [Электронный ресурс]. URL: <https://biz30.timedoctor.com/ru/c%D0%B8%D1%81%D1%82%D0%B5%D0%BC%D0>

%B0-%D0%BA%D0%BE%D0%BD%D1%82%D1%80%D0%BE%D0%BB%D1%8F-%D0%B2%D0%B5%D1%80%D1%81%D0%B8%D0%B9/. Режим доступа: свободный, дата обращения 14.12.2020.

6. Chacon, S. Pro GIT / S. Chacon, B. Straub [Электронный ресурс]. 2020. URL: <https://git-scm.com/book/ru/v1/Введение-О-контроле-версий>. Режим доступа: свободный, дата обращения 14.12.2020.

7. Collins-Sussman, B. Управление версиями в Subversion Для Subversion 1.4 / B. Collins-Sussman, B.W. Fitzpatrick, C.M. Pilato (Соответствует редакции 6037) [Электронный ресурс]. 2007. URL: <http://svnbook.red-bean.com/nightly/ru/svn-book.html>. Режим доступа: свободный, дата обращения 14.12.2020.

8. Stallman, R.M. Программа управления компиляцией GNU make Версия 3.79 / R.M. Stallman, R. McGrath, перевод (С) В. Игнатов [Электронный ресурс]. 2000. URL: http://www.linuxlib.ru/prog/make_379_manual.html#SEC1. Режим доступа: свободный, дата обращения 14.12.2020.

9. Building and Using DLLs [Электронный ресурс]. URL: <https://cygwin.com/cygwin-ug-net/dll.html>. Режим доступа: свободный, дата обращения 14.12.2020.

10. Building Windows DLLs with MinGW [Электронный ресурс]. URL: <http://www.transmissionzero.co.uk/computing/building-dlls-with-mingw/>. Режим доступа: свободный, дата обращения 14.12.2020.

11. Difference between GIT and CVS [Электронный ресурс]. 2009. URL: <https://stackoverflow.com/questions/802573/difference-between-gitand-cvs>. Режим доступа: свободный, дата обращения 14.12.2020.

12. FreeBSD Manual Pages [Электронный ресурс]. 2020. URL: <http://www.freebsd.org/cgi/man.cgi?query=make&sektion=1>. Режим доступа: свободный, дата обращения 14.12.2020.

13. Git и командная разработка (для чайников) [Электронный ресурс]. 2019. URL: <https://habr.com/ru/post/440816/>. Режим доступа: свободный, дата обращения 14.12.2020.

14. Git: советы новичкам – часть 1 [Электронный ресурс]. 2017. URL: <https://habr.com/ru/company/playrix/blog/345732/>. Режим доступа: свободный, дата обращения 14.12.2020.

15. GNU make [Электронный ресурс]. URL: <http://www.gnu.org/software/make/manual/make.html>. Режим доступа: свободный, дата обращения 14.12.2020.

СОДЕРЖАНИЕ

Предисловие	3
Лабораторная работа № 1. Командная строка и особенности использования набора компиляторов и утилит GCC	4
Лабораторная работа № 2. Особенности использования отладчика GDB	17
Лабораторная работа № 3. Создание динамических библиотек при помощи набора компиляторов и утилит GCC и их применение	27
Лабораторная работа № 4. Особенности работы с программой для управления компиляцией	35
Лабораторная работа № 5. Особенности использования систем контроля версий	41
Библиографический список	46

*Скулябина Ольга Владимировна, Логунова Татьяна Викторовна,
Мажайцев Евгений Александрович*

Компьютерный практикум

Редактор *Г.М. Звягина*

Корректор *Л.А. Петрова*

Компьютерная верстка: *А.В. Мецгерякова*

Подписано в печать 20.04.2021. Формат 60х84/16. Бумага документная.

Печать трафаретная. Усл. п. л. 2,8. Тираж 100 экз. Заказ № 39.

Балтийский государственный технический университет

Типография БГТУ

190005, С.-Петербург, 1-я Красноармейская ул., д. 1